

FS_GPLib: Breaking the Web-Scale Barrier — A Unified Acceleration Framework for Graph Propagation Models

Chang Guo
University of Science and Technology
of China
Hefei, Anhui, China
guochangcc@mail.ustc.edu.cn

Juyuan Zhang
University of Science and Technology
of China
Hefei, Anhui, China
zhangjuyuan2020@mail.ustc.edu.cn

Chang Su
University of Electronic Science and
Technology of China
Chengdu, Sichuan, China
suchang@std.uestc.edu.cn

Tianlong Fan*
University of Science and Technology
of China
Hefei, Anhui, China
tianlong.fan@ustc.edu.cn

Linyuan Lü*
University of Science and Technology
of China
Hefei, Anhui, China
linyuan.lv@ustc.edu.cn

Abstract

Propagation models are essential for modeling and simulating dynamic processes such as epidemics and information diffusion. However, existing tools struggle to scale to large-scale graphs that emerge across social networks, epidemic networks and so on, due to limited algorithmic efficiency, weak scalability, and high communication overhead. We present **FS_GPLib**, a unified library that enables efficient, high-fidelity propagation modeling on Web-scale graphs. FS_GPLib introduces a dual-acceleration framework: it combines micro-level synchronous message-passing updates with macro-level batched Monte Carlo simulation, leveraging high-dimensional tensor operations for parallel execution. To further enhance scalability, it supports distributed simulation via a novel target-node-based graph partitioning strategy that minimizes communication overhead while maintaining load balance. Theoretically, we show that under ideal assumptions, the runtime of simulations converges approximately to a constant. Extensive experiments demonstrate up to **35,000× speedup** over standard libraries such as NDLib and execution of a full Monte Carlo simulation on a Web-scale (billion-edge) graph in **11 seconds** while maintaining high simulation fidelity. FS_GPLib supports 29 propagation models—including epidemic and opinion dynamics and dynamic network models—and offers a lightweight Python API compatible with mainstream data science ecosystems. By addressing the unique challenges of modeling diffusion and cascades on the Web, FS_GPLib provides a scalable, extensible, and theoretically grounded solution for large-scale propagation analysis in epidemiology, social media analysis, and online network dynamics. Code available at: https://github.com/Allen-Ciel/FS_GPLib.

CCS Concepts

• Information systems → Data management systems.

*Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792385>

Keywords

Propagation Model; Acceleration Effects; Distributed Computing; Python Library; Message Passing

ACM Reference Format:

Chang Guo, Juyuan Zhang, Chang Su, Tianlong Fan, and Linyuan Lü. 2026. FS_GPLib: Breaking the Web-Scale Barrier — A Unified Acceleration Framework for Graph Propagation Models. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792385>

Resource Availability:

The source code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.18315461>.

1 Introduction

Propagation models are fundamental tools for understanding dynamic processes on complex networks, from epidemic outbreaks to information diffusion and opinion dynamics. On the Web—across social networks, microblogs, forums, and content-sharing platforms—these processes govern cascades, virality, and exposure. Accurate simulation of these models enables effective analysis and informed decision-making across various fields such as epidemiology [7, 16], social media analysis [6, 24], and financial risk assessment [1, 14].

However, graphs routinely reach billions of edges on the Web, efficiently simulating propagation processes at this scale becomes computationally prohibitive. Existing simulation methods suffer inherent trade-offs between accuracy, efficiency, and scalability. Classical propagation models, such as Independent Cascade (IC) [9, 10], Susceptible-Infected-Recovered (SIR) [27], and Linear Threshold (LT) [13], entail repeated edge traversals and state updates, severely limiting their efficiency on large-scale data. Approximation techniques such as mean-field [17] or τ -leaping [8] improve speed but sacrifice fidelity. Massively parallel methods [3, 22] alleviate computational bottlenecks but still rely heavily on iterative node-level processing and fail to overcome fundamental scalability limitations. Integrated propagation algorithm libraries like NDLib [30], EoN [26], and CyNetDiff [29] provide convenient interfaces but do not fundamentally resolve these efficiency and scalability

constraints. General-purpose systems for large-scale graph analytics (e.g., GraphLab [11, 12], Pregel [25], Giraph [2]) provide scalable functions but lack propagation-specific optimizations. Consequently, the fundamental performance bottleneck stemming from extensive iterative loops persists, posing significant challenges to efficient and accurate simulations on Web-scale networks.

To overcome these challenges, we propose **Faster and more Scalable python library for Graph Propagation models (FS_GPlib)**, a unified and general-purpose simulation library explicitly designed to break the Web-scale barrier in graph propagation modeling. FS_GPlib adopts a dual-acceleration framework: (i) micro-level synchronous updates via message-passing computations reduce iterative operations; (ii) macro-level batched Monte Carlo simulations leverage tensor operations for parallel acceleration. Furthermore, a distributed scheme based on target-node partitioning balances workloads and minimizes inter-process communication. It provides 29 built-in models covering epidemic, opinion, and dynamic network scenarios, with an intuitive Python API for seamless integration into data science workflows. As an open-source, extensible, and well-documented framework, FS_GPlib enables efficient large-scale propagation simulations with wide applicability in epidemiology, network science, social influence, and decision-making.

Extensive experiments demonstrate the superior performance and scalability of FS_GPlib. Our framework completes a full Monte Carlo simulation on billion-edge networks within **~11 seconds**, marking a substantial advancement in propagation model simulations. It achieves up to **35,000× speedup** over popular standard libraries such as NDlib [30] on graphs with tens of millions of edges, while maintaining high simulation fidelity.

The remainder of this paper is organized as follows. Section 2 reviews the fundamentals of graph structures and classical propagation models. Section 3 presents the theoretical foundations of our approach. Section 4 details the architecture and API design of FS_GPlib. Section 5 reports extensive experiments validating the accuracy, efficiency, and scalability of our framework. Section 6 concludes the paper and outlines directions for future work.

2 Preliminaries

In this section, we introduce essential concepts related to graphs, the data format adopted in our framework, and the fundamental paradigm of mechanism-based propagation models.

Definition 2.1 (Graph). A graph is $G = (V, E)$ with node set V and edge set E . An edge $e \in E$ is either a tuple (u, v) or a triple (u, v, w) , where u and v denote the source and target nodes, and w is an optional edge weight. Graphs with weights are weighted; otherwise unweighted.

Definition 2.2 (Compressed Sparse Row, CSR). The CSR format compactly represents a sparse adjacency matrix by storing only nonzero elements. It consists of three arrays (row pointers, column indices, and values) of length proportional to $|E|$. For unweighted graphs, the values array can be omitted to save memory.

We store graphs in CSR format, which integrates efficiently with message passing for node state updates. Compared to adjacency

matrices (which suffer from quadratic space complexity) and adjacency lists (which do not support matrix operations), CSR offers a scalable compromise for large-scale simulations.

We now define the formal paradigm of *mechanism-based propagation models*, which is central to this study.

Definition 2.3 (Mechanism-Based Propagation Model). A propagation model governed by predefined rules and parameters, independent of historical data.

These models originated from epidemiology and sociology [5, 20], with classical examples such as the SIR and IC models. More recent extensions incorporate probabilistic rules and structural heterogeneity [19, 23, 33].

Definition 2.4 (Paradigm of Mechanism-Based Propagation Model). A mechanism-based propagation model consists of:

- A graph structure $G = (V, E)$.
- A node state space Σ .
- A probability space Ω representing all random variables and parameters (e.g., activation probabilities, thresholds).
- A discrete-time update sequence $\{(k, V_k) \mid k = 0, 1, \dots, K - 1\}$, where $V_k \subseteq V$ is the set of nodes updated at step k , determined deterministically or sampled from Ω .
- A node update function $F_v : \Sigma^{N^-(v) \cup \{v\}} \times \Omega \rightarrow \Sigma$, which maps the previous-round states of node v and its in-degree neighbors $N^-(v)$ to the updated state.

Given initial states $\{X_{v,0} \in \Sigma \mid v \in V\}$ and random factors $\omega \in \Omega$, the propagation process proceeds iteratively. At each step k , for each $v \in V_k$, the state is updated as:

$$X_{v,k} = F_v(X_{v,k-1}, \{X_{u,k-1} : u \in N^-(v)\}, \omega). \quad (1)$$

States of nodes not in V_k remain unchanged.

The implementation of mechanism-based propagation models typically involves four nested loops (Algorithm 1). The outermost loop involves Monte Carlo simulations to estimate outcomes under probabilistic conditions $\omega \in \Omega$. Within each simulation, the model iterates over K discrete steps, updating designated nodes at each step. Updating a node further involves iterating over its neighbors or edges to compute their influence. As a result, increasing the number of simulations, time steps, nodes, or edges leads to a significant increase in computational cost.

3 Theoretical Foundations of FS_GPlib

This section details the theoretical grounding of the dual-accelerated propagation framework and distributed simulation methods introduced by FS_GPlib. We illustrate their implementations via two representative propagation models: the SIR and Hegselmann-Krause (HK) models in Appendix A.

3.1 Dual-Acceleration Framework via Message Passing

Propagation models typically require numerous time steps and Monte Carlo runs for tracking node states over time and estimating outcomes. FS_GPlib addresses computational bottlenecks through a dual-acceleration strategy that combines micro-level synchronous node-state updates and macro-level batched Monte Carlo computations (Fig. 1).

Algorithm 1: Traditional mechanism-based propagation

Input: $G = (V, E)$, time steps K , Monte Carlo simulations S
Output: $\mathcal{X}^K = \{X_s^K\}_{s=1}^S$

```

1 Initialize an empty container  $\mathcal{X}^K$ ;
2 for  $s \leftarrow 1$  to  $S$  do
3   Initialize states of nodes  $X^0 = \{x_i^0 \mid i \in V\}$ ;
4   for  $k \leftarrow 0$  to  $K-1$  do
5      $X^{k+1} \leftarrow X^k$ ;
6     for  $i \in V$  do
7       Decide whether  $x_i^k$  needs to update;
8       if update is required then
9         Use  $\{x_j^k \mid j \in N(i)\}$  or a random probability;
10        Update  $x_i^{k+1}$  accordingly;
11      else
12         $x_i^{k+1} \leftarrow x_i^k$ ;
13    Record  $X^{k+1}$ ;
14  Append  $X^K$  to  $\mathcal{X}^K$ ;
15 return  $\mathcal{X}^K$ 

```

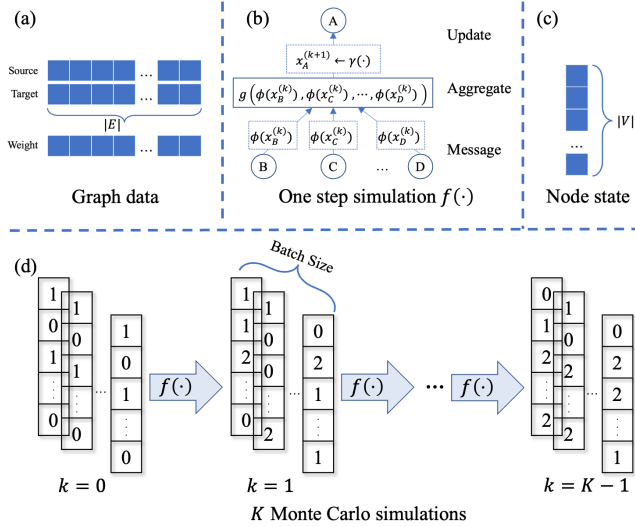


Figure 1: Architecture of the dual-acceleration propagation framework based on micro-level message passing and macro-level batched Monte Carlo computations.

Micro-level Acceleration via Node-Synchronous Computation. Traditional propagation algorithms repeatedly traverse nodes and their neighbors, resulting in significant computational redundancy. To implement efficiently, we rely on the following observation about the locality of node activation.

Observation 1. In the mechanism propagation model, the evolution pattern of nodes is similar to the forward propagation paradigm of the Message Passing framework, including message passing, aggregation, and state updating (Fig. 1(b)).

We propose a micro-level node-synchronous computation method that aggregates neighbor information for every node at once and updates all node states simultaneously, thereby eliminating redundant traversals and enhancing computational efficiency.

Definition 3.1 (Node-Synchronized Update Method). Let $x_i^{(k)} \in \Sigma$ denote the state of node i at step k , and $\omega^{(k)}$ represent the random component at step k . At each time step, source nodes transmit messages to destination nodes, which then aggregate the received messages and update their states as:

$$x_i^{(k)} = \gamma \left(x_i^{(k-1)}, g \left(\phi(x_i^{(k-1)}, x_j^{(k-1)}, e_{j,i}, \omega^{(k)}), \omega^{(k)} \right) \right), \quad (2)$$

where functions $\phi(\cdot)$, $g(\cdot)$ and $\gamma(\cdot)$ encode message passing, aggregation, and node-state updating, respectively.

In propagation models, nodes are represented by one-dimensional features, and some state transitions depend solely on probabilities rather than neighbor states. Therefore, in simulating a single time step of the propagation model, the states of neighboring nodes are aggregated first:

$$m_{ji}^{(k)} = \phi \left(x_i^{(k-1)}, x_j^{(k-1)}, e_{j,i}, \omega^{(k)} \right), \quad (3)$$

function $\phi(\cdot)$ is defined according to the propagation-model mechanism and typically takes edge information, the states of the source and target nodes from the previous time step, and the random element sampled at this step or initially. Then aggregate the received information:

$$m_i^{(k)} = g \left(\{m_{ji}^{(k)} \mid j \in N(i)\} \right), \quad (4)$$

the aggregated information $m_i^{(k)}$ is used to update the node status:

$$x_i^{(k)} = \gamma \left(x_i^{(k-1)}, m_i^{(k)}, \omega^{(k)} \right), \quad (5)$$

during the node state update step, the update method for each target node is determined based on its current state $x_i^{(k-1)}$. It may be updated using neighbor information $m_i^{(k)}$, or, if its update is independent of neighbors, via other parameters.

Micro-level acceleration is achieved by performing the above operations within tensor computations to compute all nodes in parallel, reducing interpreter overhead and exploiting CPU/GPU parallelism.

Macro-level Acceleration via Batched Monte Carlo Simulations. Monte Carlo simulations repeatedly execute identical algorithms on the same graph. Inspired by deep learning batch processing, we propose:

Definition 3.2 (Batched Monte Carlo Method). Node states from B Monte Carlo simulations are stacked into a tensor $\mathcal{X} \in \mathbb{R}^{B \times |V|}$, enabling parallel propagation computations:

$$f(\mathcal{X}, \{\omega\}^B), \quad (6)$$

where each simulation within the batch independently samples randomness ω .

Figure 1(d) illustrates the batch-parallel method. In this approach, multiple initial node-state vectors are stacked into a high-dimensional tensor, and forward propagation is executed as a single batched

operation across K discrete time steps. This enables simultaneous execution of multiple Monte Carlo simulations through vectorized operations.

Compared to conventional multi-process approaches—which distribute Monte Carlo simulations across CPU cores—batch parallelism consolidates multiple node states into a single high-dimensional tensor, enabling all runs to be executed concurrently within one forward-propagation pass. This design avoids repeated model instantiation and redundant operator invocations, yielding higher computational efficiency. Proper tuning of the batch size also ensures better GPU utilization.

By integrating both micro-level (synchronous node updates) and macro-level (batched Monte Carlo) acceleration, the proposed framework significantly reduces simulation runtime while preserving the time-step iteration logic. This enables fine-grained tracking of node-state evolution over time and efficient estimation of the expected propagation range.

3.2 Distributed Simulation via Target-node-based Partitioning

Simulating propagation on Web-scale graphs poses significant challenges in memory consumption and computational efficiency. Partitioning strategies are typically categorized into *vertex partitioning* and *edge partitioning*. Vertex partitioning methods, such as METIS [18], aim to minimize cross-partition communication, while edge partitioning emphasizes load balancing and reducing vertex replication.

FS_GPlib adopts a distributed simulation strategy that partitions the graph across multiple processes based on target nodes, aiming to balance computational loads while minimizing communication overhead (Fig. 2). In propagation models, target nodes aggregate messages from source nodes to update their state. If a target spans multiple partitions, inter-process communication is needed to exchange partial messages, adding cost and complexity. FS_GPlib avoids this by assigning each target node to exactly one partition, enabling local aggregation and updates without cross-partition messaging. After local updates, an `all_reduce` synchronizes target-node states across processes, and the updated global state is broadcast for the next iteration—ensuring correctness while minimizing redundant communication.

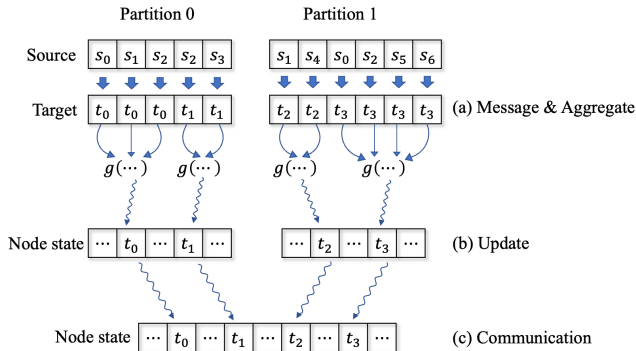


Figure 2: Distributed propagation simulation strategy with target-node-based partitioning.

Definition 3.3 (Target-Node-Based Partitioning via Longest Processing Time Heuristic). Sort nodes by non-increasing in-degree to obtain the permutation $\pi = (\pi_1, \pi_2, \dots, \pi_N)$. Let the prefix sum of weights be $C_i = \sum_{n=1}^i w_{\pi_n}$ and the total weight $W = C_N$. Each node π_i is assigned to partition

$$q(i) = \left\lceil \frac{dC_i}{W} \right\rceil, \quad q(i) \in \{1, \dots, d\}, \quad (7)$$

where d is the number of partitions. Define the partitioned target-node sets and induced edge sets by

$$P_q = \{\pi_i \mid q(i) = q\}, \quad E_q = \{(u, v) \in E \mid v \in P_q\}. \quad (8)$$

This guarantees every target node belongs to exactly one partition, and each process handles a subgraph with approximately balanced edge load.

PROPOSITION 3.4 (LOAD BALANCE GUARANTEE). Let $w_{\max} = \max_j w_j$. For each partition q , the edge load satisfies

$$\left| \sum_{i \in P_q} w_{\pi_i} - \frac{W}{d} \right| < w_{\max}, \quad (9)$$

thus the workload deviation from the ideal average W/d is bounded by at most the maximum in-degree.

Remark 1 (Communication Efficiency). Since each target node appears in exactly one partition, its state can be updated locally without inter-process communication. Consequently, at each time step, only the updated target-node states need to be communicated across processes, yielding a communication complexity of $O(|V|)$ per step, independent of the total number of edges $|E|$.

3.3 Theoretical Analysis

A key theoretical contribution of FS_GPlib lies in its theoretically grounded ability to achieve near-constant runtime complexity through carefully designed micro-level synchronous node-state updates and macro-level batch-parallel Monte Carlo execution.

In line with most real-world graphs, we assume that the number of edges scales linearly with the number of nodes, i.e., $m = \Theta(n)$. We consider discrete-time propagation models, where each node updates its state in constant time based on neighbor states and random variables.

THEOREM 3.5 (CONSTANT-TIME MICRO-LEVEL NODE SYNCHRONIZATION). Let $G = (V, E)$ be a sparse graph with $|V| = n$ nodes and $|E| = m = \Theta(n)$ edges. Consider a synchronous node-update rule (Eq. 2) under a message-passing-based propagation model, where each node's update is computed from its immediate neighbors' states or random variables.

Assuming an ideal parallel execution environment where $p = \Theta(m)$ processing units can concurrently handle all m edges without sequential fallback, and runtime overhead is negligible, the runtime per synchronous update step is:

$$T_{\text{step}} = O(1). \quad (10)$$

COROLLARY 3.6 (CONSTANT-TIME MACRO-LEVEL BATCHED SIMULATION). Building on Theorem 3.5, suppose we execute S independent Monte Carlo simulations simultaneously via batched tensor computations, where each simulation involves $K = O(1)$ propagation steps.

Assuming an ideal execution environment where the device has sufficient capacity for $B = \Theta(S)$ copies of data to run simultaneously, and ignoring the runtime overhead, the overall time complexity is:

$$T_{all} = O(1). \quad (11)$$

See the Appendix B for the proofs of the above theorem and corollary.

These theoretical results explain FS_GPLib's empirically observed high efficiency and scalability: with constant-time micro-level synchronization and macro-level batching, our framework presents constant time complexity on graphs below a certain size, and enables efficient simulation on billion-scale graphs.

4 Architecture of FS_GPLib

This section introduces the architecture, usage, and customization capabilities of FS_GPLib.

4.1 Ensemble Algorithms

FS_GPLib includes 29 built-in propagation models classified into three categories: Epidemic Models, Opinion Dynamics Models, and Dynamic Graph Models (Table 1). All models support both directed and undirected graphs, with certain models additionally supporting weighted graphs. Each model is characterized by two primary dimensions: iteration mode (synchronous vs. asynchronous) and state representation (discrete vs. continuous). In synchronous iteration, all node states are updated in a single step, whereas asynchronous iteration updates one node or subset per step. Discrete models restrict node states to finite sets, while continuous models allow states within continuous intervals.

4.2 Algorithm Usage

4.2.1 Basic Workflow. FS_GPLib relies on the *torch* and *PyG* libraries, which must be installed beforehand. Using FS_GPLib involves two steps: instantiating the model and running simulations.

To instantiate a propagation model $f(\cdot)$, specify the graph *data*, initial node set *seeds*, model parameters $\{params\}$, and computing device *device*:

$$Model = f(data, seeds, \{params\}, device). \quad (12)$$

Four simulation execution interfaces are available:

- *Model.run_iteration()*: Execute one time step from the current node state and return the updated node state.
- *Model.run_iterations(times)*: Execute multiple time steps starting from the current node state and return the final node state after all iterations.
- *Model.run_epoch(times)*: Execute multiple time steps starting from the initial state and return the final node state after all iterations.
- *Model.run_epochs(epochs, times, batch_size)*: Perform multiple Monte Carlo simulations in batches, each starting from the initial state, and return the final node states.

The provided interfaces collectively satisfy common requirements for propagation analysis. They support both fine-grained tracking of dynamics (e.g., per-iteration evolution) and large-scale statistical analysis of final outcomes (e.g., Monte Carlo estimation of spread range). We give an example of usage in the section C.

4.2.2 Distributed Partitioning for Web-scale Graphs. FS_GPLib supports Web-scale graphs using target-node partitioning (Section 3.2), which balances computational load, minimizes communication overhead, supports partitioning of both edges and one-dimensional edge features, and stores subgraph metadata to a specified path. The workflow is as follows:

- (1) Instantiate the partitioner:

$$partitioner = GraphPartitioner(data, n_parts, root). \quad (13)$$

- (2) Generate and save partitions:

$$partitioner.generate_partition(). \quad (14)$$

- (3) Load specific subgraph partitions:

$$subdata, subnodes = load_partition(root, partition_idx). \quad (15)$$

If *root* $\neq$ *None*, partitioned subgraphs and node lists are saved under the specified root path.

This partitioning strategy scales propagation to larger graphs without device constraints. Simulations may run sequentially on one node or be accelerated in parallel across multiple nodes. While FS_GPLib does not provide a built-in interface for distributed simulation workflows, a reference implementation is available. The library also includes a suite of specialized parameters and utility functions to facilitate flexible customization.

4.3 Custom Model Development

FS_GPLib includes three main classes of pre-implemented models, derived from the base classes *DiffusionModel* and *Diffusion_process*. *DiffusionModel* handles data loading, seed nodes, parameters, device configuration, and operational interfaces. *Diffusion_process* encapsulates propagation logic. Users can either directly use existing models or extend these base classes to create custom propagation algorithms tailored to specific needs.

5 Experimental Evaluation

We conduct comprehensive experiments to evaluate FS_GPLib's accuracy, efficiency, and scalability. The evaluation includes:

- (1) **Experimental Setup:** Describes the datasets, models, parameters, software, and hardware used in the experiments.
- (2) **Accuracy Validation:** Verifies the correctness of FS_GPLib's implementations by comparing simulation outcomes against established baselines.
- (3) **Efficiency Benchmarking and Acceleration Analysis:** Assesses both micro-level and macro-level acceleration by comparing runtime across implementations and analyzing the effect of batch size on simulation throughput.
- (4) **Scalability on Billion-Scale Graphs:** Demonstrates the distributed simulation capability of FS_GPLib.

5.1 Experimental Setup

We select six real-world graph datasets spanning various sizes: Cora, PubMed, Flickr, Yelp, Pokec, and Webbase (Table 2). Among them, Pokec is used solely for FS_GPLib-based simulation, and Webbase is used solely for distributed simulation.

To evaluate the accuracy and efficiency of FS_GPLib's propagation models, we compare them against several widely used or highly efficient implementations of the IC and SIR models [36]: (1) a custom

Table 1: Summary of built-in propagation models in FS_GPlib.

Model Class	Model Name	Weighted Graph Support	Iteration Mode*	State Types
Epidemic Models	SI	Yes	Synchronous Model	Discrete
	SIS	Yes	Synchronous Model	Discrete
	SIR	Yes	Synchronous Model	Discrete
	SEIR(DT)	Yes	Synchronous Model	Discrete
	SEIR(CT)	Yes	Synchronous Model	Discrete
	SEIS(DT)	Yes	Synchronous Model	Discrete
	SEIS(CT)	Yes	Synchronous Model	Discrete
	SWIR	Yes	Synchronous Model	Discrete
	Threshold	Yes	Synchronous Model	Discrete
	Kertesz Threshold	Yes	Synchronous Model	Discrete
	Independent Cascades	No	Synchronous Model	Discrete
	Profile	No	Synchronous Model	Discrete
	Profile Threshold	No	Synchronous Model	Discrete
Opinion Dynamics Models	Voter	No	Asynchronous model	Discrete
	Q-Voter	No	Asynchronous model	Discrete
	Majority Rule	No	Asynchronous model	Discrete
	Sznajd	No	Asynchronous model	Discrete
	Weighted Hegselmann-Krause	No	Synchronous Model	Continuous
	Hegselmann-Krause	No	Synchronous Model	Continuous
Dynamic Graph Models	DySI	Yes	Synchronous Model	Discrete
	DySIS	Yes	Synchronous Model	Discrete
	DySIR	Yes	Synchronous Model	Discrete
	DySEIR(DT)	Yes	Synchronous Model	Discrete
	DySEIR(CT)	Yes	Synchronous Model	Discrete
	DySEIS(DT)	Yes	Synchronous Model	Discrete
	DySEIS(CT)	Yes	Synchronous Model	Discrete
	DySWIR	Yes	Synchronous Model	Discrete
	DyThreshold	Yes	Synchronous Model	Discrete
	DyKerteszThreshold	Yes	Synchronous Model	Discrete

*: Synchronous/Asynchronous in the table refers to updating all/some nodes in one time step.

Table 2: Summary of experimental graph datasets.

Dataset	Nodes ($ N $)	Edges ($ E $)	Avg. Degree $\langle k \rangle$
Cora [35]	2,708	10,556	7.80
PubMed [35]	19,717	88,648	8.99
Flickr [37]	89,250	899,756	20.16
Yelp [37]	716,847	13,954,819	38.93
Pokec [34]	1,632,803	30,622,564	37.51
Webbase [4]	118,142,143	992,844,891	17.27

implementation using *NetworkX* (Nx-*); (2) the *NDlib* framework (Nd-*); (3) the *CyNetDiff* library for IC [29] (CND-IC); (4) a GPU-based parallel IC implementation [32](GP-IC); (5) the *EoN* library for SIR [21] (EoN-SIR), which achieved top performance in recent studies [31]; (6) a C++ implemented Python module *Graph-tool* [28] for SIR (GT-SIR).

Unless otherwise specified, the IC model is run with infection probability $\beta = 0.5$ until convergence; the SIR model uses infection probability $\beta = 0.01$, recovery rate $\lambda = 0.005$, and runs for 100 time steps; all simulations use 1,000 Monte Carlo trials. Seed nodes are selected by degree centrality (top 10%).

Baseline implementations only support CPU execution, except for GPIC. FS_GPlib supports both CPU and GPU. CPU tests were conducted on a compute node with four Intel Xeon Platinum 8260 CPUs (24 cores and 48 threads each, 2.40 GHz fundamental frequency, 1.5 TiB RAM). GPU tests used eight NVIDIA RTX 4090 GPUs (24.5 GiB each) on a dual-socket 64-core Intel Xeon Platinum 8358 (2.6 GHz) server with 503 GiB RAM. Unless otherwise specified, only one GPU is used by default. The software environment includes Python 3.10, PyTorch 2.1.2, CUDA 12.1, with `CUDA_VISIBLE_DEVICES=0`. Detailed device-level runtime performance is presented in Section D.

5.2 Accuracy Validation

This section verifies the accuracy of FS_GPlib's propagation models by comparing their outputs against established implementations. We focus on two widely studied models—IC and SIR—and defer additional results to Table 11 in Appendix E.

Figures 3 and 4 compare the Monte Carlo simulation results of FS_GPlib with two widely used implementations: *NetworkX* (Nx-*) and *NDlib* (Nd-*). Due to performance constraints, we ran 10 simulations for Nd-*, 100 for Nx-*, and 1,000 for FS-* to ensure reliable convergence. For IC, Figure 3 presents the average number of activated nodes under varying propagation probabilities; for SIR,

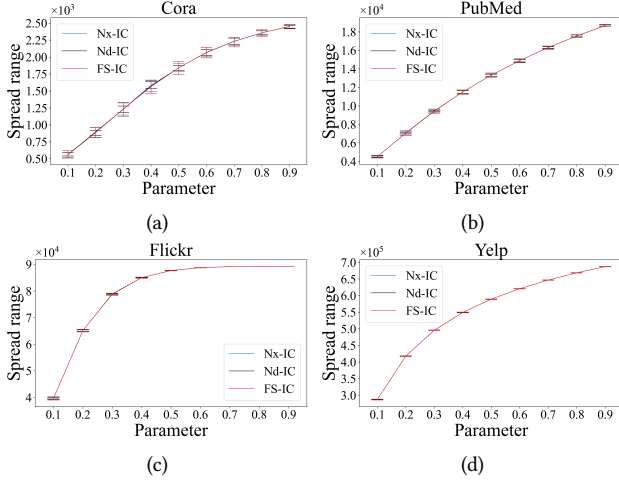


Figure 3: Monte Carlo simulation results for three IC implementations across four datasets, highlighting FS_GPlib's consistency and accuracy.

Figure 4 shows the average percentage of infection nodes across datasets. In both cases, FS_GPlib reproduces the correct dynamics: its mean trajectories align closely with those of other methods. The broader confidence bands in FS_GPlib may be attributable to the increased number of simulation samples. These results validate the fidelity of FS_GPlib's model implementations.

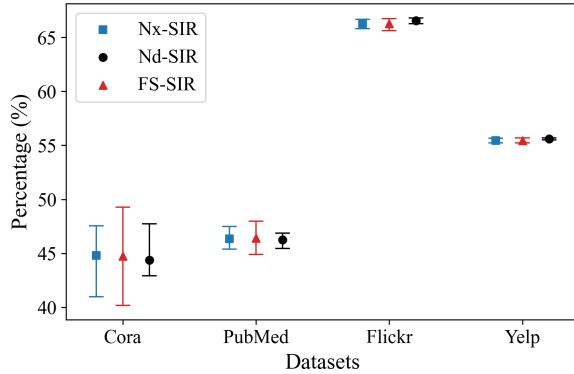


Figure 4: Monte Carlo simulation results for three SIR implementations across datasets, illustrating FS_GPlib's consistency and accuracy.

5.3 Efficiency Benchmarking and Acceleration Analysis

We evaluate the efficiency of FS_GPlib through two complementary experiments.

5.3.1 Implementation Runtime Comparison. To evaluate FS_GPlib's micro-level acceleration, we benchmark the runtime of 1,000 Monte Carlo simulations using several widely adopted implementations of the IC and SIR models: *NetworkX* (Nx-*), *NDlib* (Nd-*), *CND-IC*,

Table 3: Comparison of 1000 Monte Carlo simulations runtime performance (in seconds) among IC implementations on different datasets.

Dataset	Nx-IC	Nd-IC	CND-IC	GP-IC	FS-IC(CPU)	FS-IC(GPU)
Cora	17.79 [†]	1233.64*	0.30	1.01	5.00	3.01
PubMed	167.24 [†]	9223.87*	2.43	3.46	16.59	3.23
Flickr	1689.79 [†]	41669.81*	26.30	49.66	50.45	2.50
Yelp	24644.47 [†]	376977.50*	551.39	-	1641.92	10.75

[†]: This data is the time to perform 100 simulations multiplied by 10.

*: This data is the time to perform 10 simulations multiplied by 100.

Table 4: Comparison of 1000 Monte Carlo simulations runtime performance (in seconds) among SIR implementations on different datasets.

Dataset	Nx-SIR	Nd-SIR	EoN-SIR	GT-SIR	FS-SIR(CPU)	FS-SIR(GPU)
Cora	82.28 [†]	314.11*	17.15	6.02	60.17	27.06
PubMed	789.82 [†]	2563.64*	177.00	45.08	177.60	27.43
Flickr	8172.21 [†]	21055.04*	2240.73	168.55	898.33	27.60
Yelp	110627.07 [†]	271245.34*	28765.07 [†]	1546.66	17975.58*	99.21

[†]: This data is the time to perform 100 simulations multiplied by 10.

*: This data is the time to perform 10 simulations multiplied by 100.

GP-IC, EoN-SIR and GT-SIR. FS_GPlib is tested in both CPU and GPU modes. Runtime results are presented in Tables 3 and 4, and Figures 5 and 6 report relative speedups over the NDlib baseline.

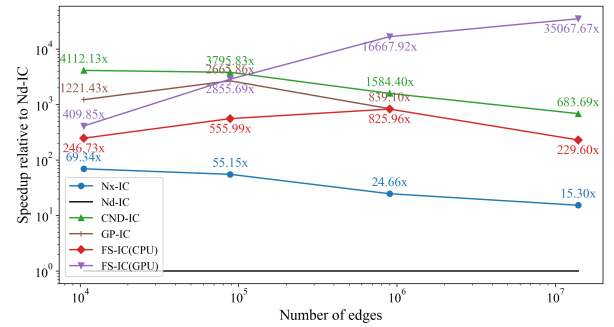


Figure 5: Speedup of IC implementations over NDlib.

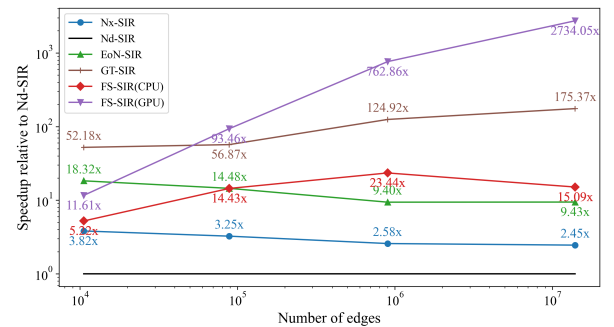


Figure 6: Speedup of SIR implementations over NDlib.

Among IC implementations, Nx-IC consistently outperforms Nd-IC but still exhibits high absolute runtimes. CND-IC performs best on small datasets; however, its efficiency gains diminish as graph size increases, indicating limited scalability. The performance of GP-IC is between CND-IC and FS-IC(CPU), but its processing of large-scale Yelp data exceeds memory, which is also the performance of limited scalability. FS-IC(CPU) delivers competitive acceleration on small to medium graphs. In contrast, FS-IC(GPU) demonstrates robust scalability: its speedup relative to Nd-IC increases with dataset size, achieving over 35,000 $\times$ acceleration on Yelp dataset—the shortest runtime among all implementations.

SIR simulation results follow a similar trend. Nx-SIR and EoN-SIR perform reasonably on smaller graphs but their efficiency drops as scale grows. FS-SIR(CPU) overtakes EoN-SIR on large datasets. GT-SIR runs the fastest on Cora data, but does not perform as well as FS-SIR(GPU) on larger datasets. Notably, the performance benefit of FS-SIR(GPU) significantly magnifies with graph size.

These results confirm FS_GPlib's strong micro-level acceleration capabilities, particularly its ability to scale efficiently on large networks by leveraging parallel hardware.

5.3.2 Batch Size and Runtime Scaling. We next evaluate FS_GPlib's macro-level acceleration by varying the batch size—i.e., the number of parallel Monte Carlo simulations executed per GPU pass. In theory, increasing batch size should linearly reduce total runtime due to amortized computation overhead. However, empirical results (Tables 5 and 6) reveal diminishing returns beyond moderate sizes, especially on large-scale graphs where hardware bottlenecks begin to dominate.

COROLLARY 5.1 (OPTIMAL BATCH REGIME). *There exists an upper bound on batch size $B_{\max}$ such that for any $B > B_{\max}$, the runtime of a single Monte Carlo simulation ceases to decrease and may even increase due to memory bandwidth saturation and GPU kernel scheduling overhead.*

This observation provides practical guidance for selecting batch sizes in large-scale simulations: choosing B close to $B_{\max}$ ensures maximal parallelism without incurring resource contention.

Table 5: FS-IC Runtime of 1000 Monte Carlo simulations under varying batch sizes.

batch size	1	10	100	1000
Cora	3.01	0.86	0.55	0.53
PubMed	3.23	0.91	0.61	0.61
Flickr	2.50	0.61	1.10	1.30
Yelp	10.75	12.32	12.84	-
Pokec	23.27	25.97	-	-

5.4 Distributed Propagation on Web-Scale Graphs

We evaluate FS_GPlib's scalability on a Web-scale graph—Webbase, containing over 118 million nodes and 990 million edges. The experiment was conducted on a single machine with eight GPUs (see Section 5.1), of which six were used for computation.

Table 6: FS-SIR Runtime of 1000 Monte Carlo simulations under varying batch sizes.

batch size	1	10	100	1000
Cora	27.06	3.29	0.86	0.66
PubMed	27.43	3.32	1.02	1.24
Flickr	27.60	4.82	6.32	6.60
Yelp	99.21	94.99	92.45	-
Pokec	229.82	216.02	-	-

As shown in Table 7, partitioning the graph required only 42 seconds. We then executed 1,000 Monte Carlo simulations across six processes, completing all runs in approximately 1.15×10^4 seconds—averaging just 11.5 seconds per full-scale simulation. The final infection ratio converged to 29.63%.

Table 7: Distributed SIR simulation result on the Webbase of 1000 Monte Carlo simulations.

Dataset	Time of partition(s)	Time of simulation(s)	Expected infection rate(%)
Webbase	42.08	1.15E+04	29.63

These empirical results corroborate Theorem 3.5 and Corollary 3.6, confirming that FS_GPlib achieves near-constant amortized runtime per simulation as graph size increases. This marks a breakthrough: FS_GPlib reduces the perceived $O(m)$ complexity of traditional agent-based simulation to practical constant time per run in theory and enables near-real-time propagation modeling on billion-scale graphs empirically, unlocking practical research and engineering applications at unprecedented scale.

6 Summary

We introduced **FS_GPlib**, a unified Python framework for fast, scalable propagation simulations on large graphs. By integrating a dual-accelerated design—message-passing-based agent modeling and batched execution, and a distributed propagation simulation method, FS_GPlib bridges the longstanding gap between efficiency and scalability in propagation modeling.

The framework achieves significant improvements over existing libraries, delivering up to **35,000 $\times$ speedup** and enabling simulation on graphs with over **100 million nodes**. Our design is supported by a new theoretical result: under GPU-accelerated batched Monte Carlo sampling, the amortized runtime per simulation is **asymptotically constant** under realistic batching and parallelism assumptions, even on web-scale networks. This formally explains FS_GPlib's near-constant wall-clock time in practice and validates its scalability guarantee. FS_GPlib lays the foundation for high-throughput, high-fidelity, and theoretically grounded propagation modeling at scale.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (Grant Nos. T2293771, 62503447).

References

- [1] Daron Acemoglu, Asuman Ozdaglar, and Alireza Tahbaz-Salehi. 2015. Systemic risk and stability in financial networks. *American Economic Review* 105, 2 (2015), 564–608. doi:10.1257/aer.20130456
- [2] Apache Software Foundation. 2020. Giraph - Welcome To Apache Giraph! Apache Giraph Official Website. <https://giraph.apache.org/> Accessed: 2026-01-24..
- [3] Christopher L Barrett, Keith R Bisset, Stephen G Eubank, Xizhou Feng, and Madhav V Marathe. 2008. Episimdemics: an efficient algorithm for simulating the spread of infectious disease over large realistic social networks. In *SC'08: Proceedings of the 2008 ACM/IEEE Conference on Supercomputing*. IEEE, 1–12. doi:10.1109/SC.2008.5214892
- [4] Paolo Boldi and Sebastiano Vigna. 2004. The webgraph framework I: compression techniques. In *Proceedings of the 13th international conference on World Wide Web (WWW '04)*. Association for Computing Machinery, New York, NY, USA, 595–602. doi:10.1145/988672.988752
- [5] Daryl J Daley and David G Kendall. 1964. Epidemics and rumours. *Nature* 204, 4963 (1964), 1118–1118. doi:10.1038/2041118a0
- [6] Ravi Kishore Devarapalli and Anupam Biswas. 2024. Estimating rumor source in social networks using incomplete observer information. *Expert Systems with Applications* 249 (2024), 123499. doi:10.1016/j.eswa.2024.123499
- [7] Moumita Ghosh, Samhita Das, and Pritha Das. 2022. Dynamics and control of delayed rumor propagation through social networks. *Journal of Applied Mathematics and Computing* (2022), 1–30. doi:10.1007/s12190-021-01643-5
- [8] Daniel T Gillespie. 2001. Approximate accelerated stochastic simulation of chemically reacting systems. *The Journal of Chemical Physics* 115, 4 (2001), 1716–1733. doi:10.1063/1.1378322
- [9] Jacob Goldenberg, Barak Libai, and Eitan Muller. 2001. Talk of the network: A complex systems look at the underlying process of word-of-mouth. *Marketing letters* 12 (2001), 211–223. doi:10.1023/A:1011122126881
- [10] Jacob Goldenberg, Barak Libai, and Eitan Muller. 2001. Using complex systems analysis to advance marketing theory development: Modeling heterogeneity effects on new product growth through stochastic cellular automata. *Academy of Marketing Science Review* 9, 3 (2001), 1–18.
- [11] Joseph E Gonzalez, Yucheng Low, Haijie Gu, Danny Bickson, and Carlos Guestrin. 2012. PowerGraph: Distributed Graph-Parallel Computation on Natural Graphs. In *10th USENIX symposium on operating systems design and implementation (OSDI 12)*. USENIX Association, Hollywood, CA, 17–30.
- [12] Joseph E Gonzalez, Reynold S Xin, Ankur Dave, Daniel Crankshaw, Michael J Franklin, and Ion Stoica. 2014. GraphX: Graph processing in a distributed dataflow framework. In *11th USENIX symposium on operating systems design and implementation (OSDI 14)*. 599–613.
- [13] Mark Granovetter. 1978. Threshold models of collective behavior. *American journal of sociology* 83, 6 (1978), 1420–1443. doi:10.1086/226707
- [14] Andrew G Haldane and Robert M May. 2011. Systemic risk in banking ecosystems. *Nature* 469, 7330 (2011), 351–355. doi:10.1038/nature09659
- [15] Rainer Hegselmann and Ulrich Krause. 2002. Opinion dynamics and bounded confidence: models, analysis and simulation. *Journal of Artificial Societies and Social Simulation* 5, 3 (2002).
- [16] Ankur Jain, Joydip Dhar, and Vijay K Gupta. 2023. Optimal control of rumor spreading model on homogeneous social network with consideration of influence delay of thinkers. *Differential Equations and Dynamical Systems* 31, 1 (2023), 113–134. doi:10.1007/s12591-019-00484-w
- [17] Samuel M Jenness, Steven M Goodreau, and Martina Morris. 2018. EpiModel: an R package for mathematical modeling of infectious disease over networks. *Journal of Statistical Software* 84 (2018), 1–47. doi:10.18637/jss.v084.i08
- [18] George Karypis and Vipin Kumar. 1998. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing* 20, 1 (1998), 359–392. doi:10.1137/S1064827595287997
- [19] David Kempe, Jon Kleinberg, and Éva Tardos. 2003. Maximizing the spread of influence through a social network. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining (Washington, D.C.) (KDD '03)*. 137–146. doi:10.1145/956750.956769
- [20] William Ogilvy Kermack and Anderson G McKendrick. 1927. A contribution to the mathematical theory of epidemics. *Proceedings of the Royal Society of London. Series A, Containing Papers of a Mathematical and Physical Character* 115, 772 (1927), 700–721. doi:10.1098/rspa.1927.0118
- [21] István Z Kiss, Joel C Miller, and Péter L Simon. 2017. *Mathematics of Epidemics on Networks*. Springer, Cham. doi:10.1007/978-3-319-50806-1
- [22] Till Köster, Leon Herrmann, Philipp Andelfinger, and Adelinde Uhrmacher. 2022. GPU-Accelerated Simulation Ensembles of Stochastic Reaction Networks. In *2022 Winter Simulation Conference (WSC)*. IEEE, 2570–2581. doi:10.1109/WSC57314.2022.10015448
- [23] Weimin Li, Yaqiong Li, Wei Liu, and Can Wang. 2022. An influence maximization method based on crowd emotion under an emotion-based attribute social network. *Information Processing & Management* 59, 2 (2022), 102818. doi:10.1016/j.ipm.2021.102818
- [24] Chen Ling, Junji Jiang, Junxiang Wang, My T Thai, Renhao Xue, James Song, Meikang Qiu, and Liang Zhao. 2023. Deep graph representation learning and optimization for influence maximization. In *International conference on machine learning*. PMLR, 21350–21361.
- [25] Grzegorz Malewicz, Matthew H Austern, Aart JC Bik, James C Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. 2010. Pregel: a system for large-scale graph processing. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data (Indianapolis, Indiana, USA) (SIGMOD '10)*. 135–146. doi:10.1145/1807167.1807184
- [26] Joel C Miller and Tony Ting. 2020. Eon (epidemics on networks): a fast, flexible python package for simulation, analytic approximation, and analysis of epidemics on networks. *arXiv preprint arXiv:2001.02436* (2020). doi:10.48550/arXiv.2001.02436
- [27] Romualdo Pastor-Satorras, Claudio Castellano, Piet Van Mieghem, and Alessandro Vespignani. 2015. Epidemic processes in complex networks. *Reviews of Modern Physics* 87, 3 (2015), 925–979. doi:10.1103/RevModPhys.87.925
- [28] Tiago P. Peixoto. 2014. The graph-tool python library. *figshare* (2014). doi:10.6084/m9.figshare.1164194
- [29] Eliot W Robson, Dharmath Reddy, and Abhishek K Umrawal. 2024. CyNetDiff: A Python Library for Accelerated Implementation of Network Diffusion Models. *Proceedings of the VLDB Endowment* 17, 12 (2024), 4409–4412. doi:10.14778/3685800.3685887
- [30] Giulio Rossetti, Letizia Milli, and Salvatore Rinzivillo. 2018. NDlib: a python library to model and analyze diffusion processes over complex networks. In *Companion Proceedings of the The Web Conference 2018 (Lyon, France) (WWW '18)*. 183–186. doi:10.1145/3184558.3186974
- [31] Mohammad Hossein Samaei, Faryad Darabi Sahneh, and Caterina Scoglio. 2025. FastGEMF: Scalable High-Speed Simulation of Stochastic Spreading Processes Over Complex Multilayer Networks. *IEEE Access* 13 (2025), 27112–27125. doi:10.1109/ACCESS.2025.3539345
- [32] Chang Su, Xu Na, Fang Zhou, and Linyuan Lü. 2025. GPIC: a GPU-based parallel independent cascade algorithm in complex networks. *Chinese Physics B* 34, 3 (2025), 030204. doi:10.1088/1674-1056/adb67c
- [33] Lichao Sun, Albert Chen, Philip S Yu, and Wei Chen. 2020. Influence maximization with spontaneous user adoption. In *Proceedings of the 13th International Conference on Web Search and Data Mining (Houston, TX, USA) (WSDM '20)*. 573–581. doi:10.1145/3336191.3371791
- [34] Lubos Takac and Michal Zabovsky. 2012. Data analysis in public social networks. In *International scientific conference and international workshop present day trends of innovations*, Vol. 1.
- [35] Zhilin Yang, William Cohen, and Ruslan Salakhudinov. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*, Vol. 48. PMLR, 40–48.
- [36] Mina Yousef and Caterina Scoglio. 2011. An individual-based approach to SIR epidemics in contact networks. *Journal of Theoretical Biology* 283, 1 (2011), 136–144. doi:10.1016/j.jtbi.2011.05.029
- [37] Hanqing Zeng, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, and Viktor Prasanna. 2019. Graphsaint: Graph sampling based inductive learning method. *arXiv preprint arXiv:1907.04931* (2019). doi:10.48550/arXiv.1907.04931

A Examples of Accelerated Model Implementations

We illustrate our framework using two classical models: the discrete-state SIR model and the continuous-state HK model.

A.1 Micro-accelerated implementation of SIR Model

The SIR Model assumes that infection spreads only through links between neighboring nodes in a graph $G = (V, E)$. Each node is in one of three states: S , I , or R . A susceptible node i is infected by its infected neighbors $j \in N(i)$ with rate β , while infected nodes recover with rate γ :

$$\frac{dS_i}{dt} = -\beta \sum_{j \in N(i)} S_i I_j, \quad \frac{dI_i}{dt} = \beta \sum_{j \in N(i)} S_i I_j - \gamma I_i, \quad \frac{dR_i}{dt} = \gamma I_i. \quad (16)$$

According to Figure 7, node transitions follow two rules: 1) if a S state node has I state neighbors, each I state neighbor transmits the infection to the S state node with probability β ; 2) the I state node is recovered to the R state with probability γ .

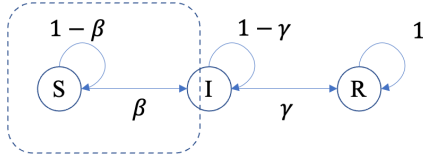


Figure 7: State transition diagram of SIR model.

Definition A.1 (Micro-accelerated implementation of SIR Model). We represent node states using two Boolean indicator vectors $h, r \in \{0, 1\}^N$, where $h_i = 1$ denotes *infected*, $r_i = 1$ denotes *recovered*, and $(h_i, r_i) = (0, 0)$ denotes *susceptible*. The update of the system at step k is decomposed into three stages:

Each infected neighbor j of node i transmits a log-probability contribution

$$m_{ji}^{(k)} = h_j^{(k-1)} \cdot \log(1 - \beta) \quad (17)$$

Node i collects contributions from all neighbors $N(i)$ to compute its infection probability

$$m_i^{(k)} = 1 - \exp\left(-\sum_{j \in N(i)} m_{ji}^{(k)}\right) \quad (18)$$

The indicator variables are updated with independent uniform random variables $U_i^{\text{inf}}, U_i^{\text{rec}} \sim \text{Uniform}(0, 1)$

$$h_i^{(k)} = \begin{cases} 1, & \text{if } (h_i^{(k-1)} = 0 \wedge r_i^{(k-1)} = 0) \wedge (U_i^{\text{inf}} < m_i^{(k)}), \\ 1, & \text{if } h_i^{(k-1)} = 1 \wedge (U_i^{\text{rec}} \geq \delta), \\ 0, & \text{otherwise,} \end{cases} \quad (19)$$

$$r_i^{(k)} = \begin{cases} 1, & \text{if } h_i^{(k-1)} = 1 \wedge (U_i^{\text{rec}} < \delta), \\ r_i^{(k-1)}, & \text{otherwise.} \end{cases}$$

After simulation, Boolean features are converted into an N -length state vector with entries 0, 1, or 2, denoting S , I , and R .

A.2 Micro-accelerated implementation of HK model

The HK model [15] describes continuous opinion dynamics. Each node holds an initial opinion $h \in [-1, 1]$ and updates it in discrete rounds. At each step: 1) ϵ neighbor identification: find the set Γ_ϵ of each node, $d_{i,j} = |h_i - h_j| \leq \epsilon, j \in \Gamma_\epsilon$; 2) update the opinion value: The opinion of node i at the next time step is updated as follows:

$$h_i^{(k)} = \frac{\sum_{j \in \Gamma_\epsilon} h_j^{(k-1)}}{\#\Gamma_\epsilon}, \quad (20)$$

where $\#\Gamma_\epsilon$ denotes the number of ϵ neighbors.

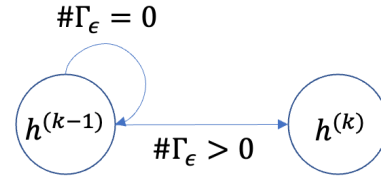


Figure 8: State transition diagram of HK model.

Definition A.2 (Micro-accelerated implementation of HK Model). Unlike the SIR model with three discrete states, the HK model uses a continuous opinion space. To avoid infinitely many states, we consider only two cases (Figure 8): if $\#\Gamma_\epsilon = 0$, the opinion remains unchanged; otherwise, it is updated to the ϵ -neighbors' average.

For each neighbor $j \in N(i)$, generate two message terms:

$$m_{ji-c}^{(k)} = \mathbf{1}\left(|h_i^{(k-1)} - h_j^{(k-1)}| < \epsilon\right) \cdot h_j^{(k-1)}, \quad (21)$$

$$m_{ji-v}^{(k)} = \mathbf{1}\left(|h_i^{(k-1)} - h_j^{(k-1)}| < \epsilon\right).$$

where $\mathbf{1}(\cdot)$ is the indicator function.

Node i aggregates received messages by averaging opinions of ϵ -neighbors:

$$m_i^{(k)} = \begin{cases} \frac{\sum_{j \in N(i)} m_{ji-c}^{(k)}}{\sum_{j \in N(i)} m_{ji-v}^{(k)}}, & \text{if } \sum_{j \in N(i)} m_{ji-v}^{(k)} > 0, \\ \text{NaN}, & \text{otherwise.} \end{cases} \quad (22)$$

Finally, the opinion of node i is updated as

$$h_i^{(k)} = \begin{cases} m_i^{(k)}, & \text{if } m_i^{(k)} \neq \text{NaN}, \\ h_i^{(k-1)}, & \text{otherwise.} \end{cases} \quad (23)$$

The node feature h , which is the node's opinion value, is returned to the user after all iterations.

Listing 1: Python example: running Monte Carlo simulations using the SIR model with FS_GPLib

```

1 | import torch
2 | from torch_geometric.datasets import Planetoid
3 | from torch_geometric.utils import degree
4 | from FS_GPLib.Epidemics.SIRModel import SIRModel
5 |
6 | def Degree_centrality_seeds(data, num_seeds):
7 |     target = data.edge_index[1]
8 |     D = degree(target, data.num_nodes)
9 |     _, index = torch.sort(D, descending=True)
10 |    seeds = index[:num_seeds]
11 |    return seeds.tolist()
12 |
13 | dataset = Planetoid(root='./dataset', name='Cora')
14 | data = dataset[0]
15 | num_seeds = int(data.num_nodes*0.1)
16 | seeds = Degree_centrality_seeds(data, num_seeds)
17 |
18 | i_beta=0.01 # infection
19 | r_lambda=0.005 # recovery
20 | d = 1 # device
21 | MC = 10000 # simulation times
22 | it = 100 # iteration times
23 | bs = 2000 # batch size
24 | model = SIRModel(data, seeds, i_beta, r_lambda, d)
25 | finals = model.run_epochs(MC, it, bs)
26 |
27 | count = (finals>0).sum().item()/MC
28 | print(f'Final spread range is {count}')
```

B Proofs

PROOF OF THEOREM 3.5. For each propagation step, the computational complexity is dominated by edge-level operations, requiring $O(m)$ work sequentially. When $p = \Theta(m)$ parallel processors can simultaneously accommodate all m edges, the per-processor workload becomes $O(1)$. Under ideal parallel execution with negligible overhead, all processors complete their assigned work in parallel within constant time, achieving:

$$T_{\text{step}} = O\left(\frac{m}{p}\right) = O(1).$$

Thus, the runtime per update step is constant. $\square$

PROOF OF COROLLARY 3.6. Running one simulation individually requires $O(1)$ time for $K = O(1)$ steps. For S serial Monte Carlo simulations, it takes $O(S)$ time. With batch-parallel execution, B simulations share tensor operations, resulting in total runtime $O(1)$ for the entire batch:

$$T_{\text{sim}} = \frac{O(S)}{B} = O\left(\frac{S}{B}\right) = O(1).$$

Hence, under batch-parallel settings, the runtime of all Monte Carlo simulations is constant. $\square$

C Examples of Use

Listing 1 demonstrates usage: loading Cora dataset, selecting seeds via degree centrality, specifying SIR model parameters, performing 10,000 Monte Carlo simulations with batch processing, and computing the final propagation range.

D Performance of Different Devices

Considering that researchers work in diverse environments, this section assesses the performance of the algorithm across different devices. We selected four platforms: Server-Linux-CPU (SLC), server-Linux-GPU (SLG), PC-Linux-GPU (PLG), and PC-MacOS-CPU (PMC). Note that PLG setup uses WSL2 to run the Linux kernel within Windows. Table 8 lists the hardware specifications and operating systems for each platform. Remaining software configurations are detailed in Section 5.1.

Table 9 and Table 10 present the runtime of IC and SIR models on four platforms using the same parameters as in Section 5.3.1 (batch size = 1). Each runtime is the average of three independent runs, and between runs we ensured device temperatures returned to normal to avoid thermal-throttling artifacts. SLG dominates overall due to superior parallel acceleration, and PLG follows closely. However, PMC outperforms both GPUs on small-scale data with minimal batch sizes where GPU utilization is inefficient. The overall running time of SLC is relatively long, especially for large-scale data.

This experiment evaluated diverse computing environments (servers vs. PCs, GPU vs. CPU, different OS). The algorithms ran successfully across all configurations with strong performance, offering practical guidance for device selection.

E Accuracy Validation of FS_GPLib

We validated FS_GPLib on Cora and PubMed, with results (Table 11) consistent across NDLlib, NetworkX, and our implementation. Algorithm settings are detailed in the code.

Due to computational constraints, not all models completed 1000 Monte Carlo simulations. Specifically, the Threshold, HK, and WHK models were executed only once. Simulation results show the expected counts of activated/infected nodes. For the HK and WHK models, nodes with a viewpoint value exceeding 0.5 were designated as activated/infected nodes. Notably, results of SWIR represent the expected sum of weak, infected, and recovered nodes. Implementation differences between some models and their NDLlib counterparts precluded their inclusion in the comparison.

These computational results empirically establish FS_GPLib’s accuracy at the experimental level, thereby providing foundational validation for the efficiency and scalability improvements demonstrated in the main text.

Table 8: Equipment information of four devices.

	SLC	SLG	PLG	PMC
CPU	Intel(R) Xeon(R) Platinum 8260 CPU @ 2.40GHz × 4	Intel(R) Xeon(R) Platinum 8358 CPU @ 2.60 GHz × 2	13th Gen Intel(R) Core(TM) i7-13700F	Apple M1 Pro
Core/Thread	24 Core/48 Thread	32 Core/64 Thread	12 Core/24 Thread	8 Core
Main frequency	2.40 GHz	2.60 GHz	2.11 GHz	-
Memory	1.5 TiB	503 GiB	7.6 GiB	16 GB
GPU	-	NVIDIA GeForce RTX 4090 × 8	NVIDIA GeForce RTX 4060 Ti × 1	-
GPU video memory	-	24.5 GiB × 8	8.0 GiB	-
Operating system/kernel	Rocky Linux 8.5	Rocky Linux 8.5	Ubuntu 22.04.5 LTS(WSL20	macOS 15.2
Other	-	CUDA_VISIBLE_DEVICES=0		

Table 9: FS-IC Runtime of 1000 Monte Carlo simulations under varying devices.

device	SLC	PLG	SLG	PMC
Cora	2.69	3.30	3.01	1.41
PubMed	10.15	3.80	3.23	5.24
Flickr	31.16	2.50	2.50	23.75
Yelp	1271.11 [†]	33.34	10.75	597.65
Pokec	2863.42 [†]	71.90	23.27	1487.83 [†]

[†]: This data is the time to perform 100 simulations multiplied by 10.

Table 10: FS-SIR Runtime of 1000 Monte Carlo simulations under varying devices.

device	SLC	PLG	SLG	PMC
Cora	30.73	26.49	27.06	12.78
PubMed	109.28	25.80	27.43	53.42
Flickr	574.71	28.01	27.60	431.03
Yelp	13776.55*	367.62	99.21	6888.00 [†]
Pokec	31129.42*	868.66 [†]	229.83	16812.12*

[†]: This data is the time to perform 100 simulations multiplied by 10.

*: This data is the time to perform 10 simulations multiplied by 100.

Table 11: Simulation results for the three implementations.

Model (parameters)	Cora			PubMed		
	NDlib	Networkx	FS_GPLib	NDlib	Networkx	FS_GPLib
SIR (0.01,0.005)	1212.70	1210.51	1211.64	9153.00	9158.47	9151.78
IC (0.5)	1842.00 [†]	1844.87 [†]	1844.791	13319.50	13319.32	13318.44
SI (0.01)	1857.50*	1866.22 [†]	1868.57	14235.8*	14306.69 [†]	14312.23
SIS (0.01,0.005)	1579.60*	1587.92 [†]	1587.012	12204.20*	12230.13 [†]	12229.61
SEIR (0.01,0.005,0.05)	1052.58 [†]	1053.95 [†]	1051.98	8202.5*	8246.28 [†]	8249.82
SEIS (0.01,0.005,0.05)	1208.57 [†]	1211.69 [†]	1213.51	9645.5*	9697.68 [†]	9684.86
SEIRct (0.01,0.005,0.05)	–	563.13 [†]	564.60	–	4215.56 [†]	4220.29
SEISct (0.01,0.0005,0.005)	–	838.74 [†]	836.60	–	6751.09 [†]	6765.04
SWIR (0.01,0.005,0.05)	–	310.33 [†]	309.41	–	2347.85 [†]	2345.32
Threshold (0.5) [‡]	2092	2092	2092	17484	17484	17484
KerteszThreshold (0.7,0.01,0.1)	2124.70*	2129.40*	2132.70	16741.30*	16727.60*	16751.35
Profile (0.5,0.01,0.1)	–	2471.40*	2473.68	–	18047.60*	18063.24
ProfileThreshold (0.7,0.25,0.01,0.1)	–	2143.70*	2143.61	–	15806.20*	15790.16
Voter	292.76	293.16	293.28	2020.52	2020.83	2020.16
Qvoter (2,0.1)	–	285.76 [†]	285.54	–	2018.51 [†]	2017.48
MajorityRule (5)	227.34 [†]	226.56 [†]	227.73	1926.12 [†]	1926.26 [†]	1925.58
Sznajd	213.085	208.51 [†]	208.09	1981.10 [†]	1973.35 [†]	1974.25
HK (0.32) [‡]	–	377	377	–	2558	2558
WHK (0.32,0.2) [‡]	–	882	882	–	6268	6268

[†]: This data is the expectation of running 100 simulations. *: This data is the expectation of running 10 simulations. [‡]: This model only perform one simulation.